Rosbot Maze Navigation – Approach

Oct 22, 2026 · @Hemanth

Group: Sai Pavan Bolagani, Hemanthkumar Muthusamy Gunasekaran, Marshal Sebastine

The Rosbot reaches the blue and then the yellow pillar in all five environments, with one codebase and one set of default parameters.

Results

All five environments are solved from the original start pose. Times are Webots R2025a simulation time on macOS, as printed by the controller in its `RESULT` block at the end of each run and visible in the videos.

Environment	Start simulation → blue pillar reached [s]	Blue pillar reached → yellow pillar reached [s]	Total [s]
Maze 1	39.49	20.61	60.10
Maze 2	40.96	14.27	55.23
Maze 3	17.86	67.55	85.41
Maze 4	25.15	20.54	45.70
Maze 5	7.94	9.95	17.89

The same code and the same default parameters are used in every environment. Repeated runs on the same computer give identical times, because the simulation is deterministic.

Overview

The controller runs one loop every 32 ms control step. It uses only the robot's own sensors: no supervisor and no knowledge of the world files.

1. **Sense:** wheel encoders, IMU, rotating RpLidar A2 (400 beams, 12 Hz), Astra RGB camera and depth camera, front infra-red sensors.
2. **Localise:** dead reckoning from encoders and IMU heading, corrected by lidar scan matching.
3. **Map:** a 2 cm occupancy grid with several layers (lidar walls, camera obstacles, green floor, camera coverage).

4. **Decide:** go to the current goal pillar if it is known, otherwise explore.
5. **Plan:** minimum-cost path on the grid, with safety margins to walls and green.
6. **Act:** pure pursuit path following with speed limits, plus recovery when stuck.

The code is split into modules: `navigator.py` (main loop), `perception.py`, `grid.py`, `localize.py`, `lidar_deskew.py`, `planner.py`, `control.py` and `params.py`.

Localisation

The heading comes directly from the IMU, which does not drift in the simulation. The position is integrated from the mean wheel-encoder travel.

- **Scan matching:** every second control step the lidar scan is aligned with the map built so far. The map is turned into a distance field (distance to the nearest mapped wall), and a coarse-to-fine search over x/y finds the shift with the smallest squared distance of the scan end points.
- **Lidar de-skew:** the RpLidar is a rotating lidar, so one range image contains beams measured up to 83 ms apart. Each beam is paired with the robot pose interpolated at its measurement time.
- **Wheel slip:** when the robot is wedged, the wheels turn but the robot does not move. The current scan is therefore also compared with a scan from about one second earlier. A correction is only accepted if it is physically possible: less motion than the encoders measured, in the same direction, at most 0.2 m per check (the check runs every 0.26 s), and the map must agree. Without these limits, rows of evenly spaced boards produced false 0.6 m jumps. While the robot is strongly tilted, encoder travel is ignored.
- **Braking:** abrupt stops pitch the robot forward, so the lidar plane hits the floor. Commands are acceleration-limited, and returns that could be floor hits are dropped.

Mapping

The map is a 2 cm grid. Each obstacle type needs a different sensor, because the lidar plane at 13 cm height misses floating and low boards.

Layer	Sensor	What it holds
Walls	Lidar	Log-odds occupancy; confirmed walls are "sticky" so grazing beams do not erase thin boards

Layer	Sensor	What it holds
Camera obstacles	Depth camera	Points between 3 cm and 24 cm height: floating walls above the lidar plane and low boards below it
Near boards	RGB + depth + lidar	Wood-coloured pixels at robot-top height where the depth camera (minimum range 0.6 m) has no value and the lidar sees nothing close: a board less than 0.6 m ahead
Front blind zone	Infra-red	Obstacles closer than the lidar's 0.2 m minimum range
Green floor	RGB camera	Green pixels projected onto the floor plane; evidence accumulates over several frames
Camera coverage	Depth camera	Where the camera has already seen the robot-top height free; unverified space costs more and is driven slower

For planning, lidar wall bands are reduced to their centre line and half a board thickness (2.5 cm) is subtracted. A few centimetres of localisation error therefore cannot close a narrow passage on the map.

Pillars

Pillars are found by HSV colour segmentation of the RGB image. The largest blue or yellow blob is taken as a candidate sighting.

- **Distance:** a sighting is accepted only if two independent cues agree: depth image, lidar beams inside the blob, or the blob width compared with the known pillar diameter. Through gaps between walls, the depth of the wall next to the gap is easily taken for the pillar's distance; the cross-check rejects that.
- **Position:** the pillar position is the densest cluster of agreeing sightings. Close sightings (up to 1.5 m) count fully, farther ones less (weight falls with distance squared), and close sightings are never pushed out of memory by many far ones.
- **Reached:** the robot centre must be within 0.30 m of the pillar centre, and the pillar must just have been ranged at its estimated place. If walls, green floor or a low board do not allow that, 0.36 m is accepted.
- **Line of sight:** a pillar only counts as seen if no mapped wall lies between robot and pillar, including a wall standing right next to the pillar.

Exploration and path planning

When the goal pillar is unknown, the robot turns once on the spot and then explores.

- **Frontiers:** exploration targets are cells at the border between floor the camera has seen and unknown space. Targets are scored by travel cost and by how much unseen area they reveal. Viewpoints of possible pillar sightings get a bonus.
- **Driving direction:** a target straight ahead is preferred over one that needs turning off sideways or back (half a turn costs 1 m of extra path). The robot follows a corridor to its end instead of switching direction at every junction.
- **Path search:** a minimum-cost path (Dijkstra) on the grid, using scikit-image `MCP_Geometric`.
- **Margins:** the robot centre keeps 13 cm from walls and 17.5 cm from green, plus a soft cost that keeps it centred in corridors. If no route exists, slightly smaller margins (12.5 cm, then 12 cm) are tried. Too-narrow passages stay blocked.
- **Trail:** cells the robot has already driven through always stay passable, so the way back cannot close.

Control and recovery

The path is followed with pure pursuit. The look-ahead distance shrinks in tight spaces.

- **Speed limits:** speed drops with wall clearance, with the heading error, and to 0.35 m/s where the path ahead has not been checked by the depth camera. Large heading errors are corrected by turning on the spot.
- **Stuck:** if the robot makes no progress for 2 s, it backs up with a sequence of escape manoeuvres and marks an obstacle in front of it.
- **Tilt reflex:** if the robot starts to tilt while driving forward (climbing a wall corner, wedging under a slab), it backs off at once and marks the obstacle.
- **Blocked at the goal:** if the robot gets stuck right next to the goal pillar and has just ranged it, the pillar counts as reached.

Parameterisation

All five environments run with the same code and the same default parameters in `navigator/params.py`. There are no per-maze parameter files and no switches that turn code parts on or off for a particular maze.

The code could load a per-maze value file (`params_<World>.json`), but none is used.

Problems we met and how we solved them

Problem	Effect	Solution
Braking pitches the robot, the lidar hits the floor	Phantom walls in the map	Acceleration limit; drop returns that could be floor hits
Pillar seen through a gap gets the wall's distance	False "reached" far from the pillar	Two independent distance cues must agree
Floating walls above the lidar plane	Robot drives into them	Depth-camera obstacle layer; slower driving where not yet checked
Boards closer than the depth camera's 0.6 m minimum range	Robot wedges under a plate	Detect wood colour where depth has no value and the lidar sees nothing close
Wheels spin while the robot is wedged	Pose drifts by more than 1 m	Scan-to-scan slip check; ignore encoders while tilted
Rows of evenly spaced boards (Maze 4)	Slip check matched the wrong board: false 0.6 m pose jump	Slip corrections must be physically possible and agree with the map
Many far, imprecise sightings of a pillar	Pillar position moved 0.7 m away from the real one	Sightings weighted by distance; close ones are kept
Junction choice at the end of a corridor (Maze 4)	Robot left the walled part into open space, 115 s to blue	Prefer exploration targets ahead in the driving direction
Driving into a wall corner	Robot climbs and rears up 50°	Tilt reflex: back off at the first clear tilt
Pillar right behind a wall	Robot tries to reach it through the wall	Line-of-sight test includes walls next to the pillar; the robot may not enter the area right around the pillar
Yellow pillar only 0.3 m from the start (Maze 4)	Too close for depth camera and lidar	Distance from blob width at close range

Tools, AI use and references

The code was generated with Claude (Anthropic LLM, claude.ai) in an iterative session. We defined the task and the rules, ran and analysed the tests in Webots, and decided which failures to fix and how. Each code file states this in its last lines. As far as we know, no code was copied from GitHub, Stack Overflow or other websites. The file `REFERENCES.md` lists, for each part of the code, the published method it implements.

Libraries: NumPy, SciPy, OpenCV, scikit-image, Webots R2025a controller API.

Main sources for the methods:

- H. Moravec, A. Elfes, "High resolution maps from wide angle sonar", IEEE ICRA 1985 (occupancy grid).
- S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*, MIT Press 2005 (log-odds mapping, likelihood field).
- E. Olson, "Real-time correlative scan matching", IEEE ICRA 2009.
- J. Zhang, S. Singh, "LOAM: Lidar Odometry and Mapping in Real-time", RSS 2014 (lidar motion compensation).
- B. Yamauchi, "A frontier-based approach for autonomous exploration", IEEE CIRA 1997.
- E. W. Dijkstra, "A note on two problems in connexion with graphs", Numerische Mathematik 1, 1959.
- D. V. Lu, D. Hershberger, W. D. Smart, "Layered costmaps for context-sensitive navigation", IEEE/RSJ IROS 2014 (inflation).
- R. C. Coulter, "Implementation of the Pure Pursuit Path Tracking Algorithm", CMU-RI-TR-92-01, 1992.
- T. Y. Zhang, C. Y. Suen, "A fast parallel algorithm for thinning digital patterns", Comm. ACM 27(3), 1984 (skeleton).
- R. Hartley, A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., 2004 (camera projection).
- R. Siegwart, I. Nourbakhsh, D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed., MIT Press 2011 (kinematics, odometry).